

ConvMemory: A Lightweight Learned Memory Reranker, a Negative Attribution Result, and a Research-Preview Conflict Editor

Taiheng Pan
github.com/pth2002

May 22, 2026

Abstract

We describe ConvMemory, a small ($\sim 3.6\text{M}$ -parameter) learned reranker for conversational long-term memory retrieval, trained with cross-encoder teacher supervision over fused dense and lexical features. On the LongMemEval memory family, ConvMemory operates above the BGE-large cross-encoder in Recall@10 at $12\text{--}47\times$ lower latency, remains within 0.025 Recall@10 of mxbai-rerank-large-v1 on Clean500 while running $28\times$ cheaper; under Stress1000 distractors the Recall@10 gap widens to 0.081 but ConvMemory still operates at $117\times$ lower latency; these LongMemEval numbers are single-run or single-seed and are reported as indicative cost-frontier evidence, not benchmark-grade. We then publish a rigorous negative attribution result on a previously claimed mechanism: a five-seed retrained ablation with paired bootstrap shows that ConvMemory’s learned temporal window is statistically significant on aggregate but *not* temporally specific, with the largest effects on hard non-temporal controls and no significant effect on multi-hop temporal queries. The honest description of the mechanism is cheap cross-encoder distillation in a fused dense+lexical feature space, not temporal-structure exploitation. We additionally release CCGE-LA, a low-amplitude conflict-aware candidate-set editor over ConvMemory, as a research preview with modest but consistent gains on supersession and stale/rescue slices on LoCoMo. All results are retrieval-stage; ConvMemory does not match mxbai-rerank-large-v1 in absolute LoCoMo MRR, and the report is single-author and not yet independently audited.

1 Introduction

Long-term agent and conversational memory systems accumulate large memory stores over time. Per-turn LLM-as-selector approaches scale poorly: their cost grows with the number of memories N , and aggressive truncation strategies discard recall. Strong cross-encoder rerankers are accurate but expensive at memory-store scale, often spending hundreds of milliseconds to several seconds per query when run over the top-500 candidates of a dense retriever.

This report studies one narrow question. How cheaply can a small learned reranker approximate cross-encoder quality on memory-style retrieval, and what mechanism is responsible for any quality it does deliver?

We make three contributions of varying maturity. We list them as contributions rather than promote them as equal main results: the first two are audited multi-seed results, while the third is an early-stage extension distributed as an alpha checkpoint.

1. **ConvMemory** (engineering contribution): a small ($\sim 3.6\text{M}$ -parameter) learned reranker with cross-encoder teacher supervision, distributed via the Hugging Face Hub. Its practical value is a cost-quality operating point: roughly cross-encoder Recall@10 on conversational memory retrieval at one to two orders of magnitude lower latency. We do not claim it

beats strong cross-encoders generally; in particular, mxbai-rerank-large-v1 dominates it on absolute LoCoMo MRR.

2. **A rigorous attribution study** (scientific contribution): showing that the originally proposed temporal-window mechanism is not load-bearing on this task. The gain is best described as cheap cross-encoder distillation in a fused dense+lexical feature space. We publish this as a negative result on a previously claimed mechanism because the engineering value (cost-effective reranking) survives the negative attribution.
3. **CCGE-LA** (research preview): a low-amplitude conflict-aware candidate-set editor over ConvMemory. It shows modest but consistent gains on stale/current conflict slices on LoCoMo. The distributed checkpoint is single-seed; the aggregate gains rest on an internal 5-seed study. We list this as a contribution to make the published artifact and its limits explicit, not to claim equal maturity with the other two.

Throughout, we use the phrase “to our knowledge” rather than “first” and avoid blanket claims of novelty or state-of-the-art performance. Each section that reports a number also states what that number is *not* evidence for.

2 Related Work

Dense memory retrieval. ConvMemory is built on top of standard sentence encoders. We use MPNet [Song et al., 2020] as the default backbone for the distributed checkpoint and additionally retrain on BGE-large [Xiao et al., 2023] and E5-large [Wang et al., 2022] encoders to test backbone generalization.

Cross-encoder rerankers. The strong reranker baselines we compare against are ms-marco-MiniLM-L-6 [Reimers and Gurevych, 2021], the BGE-reranker family [Xiao et al., 2023], jina-reranker-v2-multilingual [Jina AI, 2024], and mxbai-rerank-large-v1 [Mixedbread AI, 2024]. Cross-encoder distillation into smaller students is a well-known recipe; ConvMemory inherits that approach with a fused dense+lexical feature space rather than raw text–text concatenation.

Agent memory systems. Recent agent memory frameworks such as mem0 [Mem0 Team, 2024] and Letta/MemGPT [Packer et al., 2023] motivate the practical setting: a long-running agent must retrieve relevant fragments from an accumulated memory store at every turn, often under tight latency budgets.

Benchmarks. We use LoCoMo [Maharana et al., 2024] as the primary conversational memory benchmark and LongMemEval [Wu et al., 2024] for cost-frontier measurements. External out-of-distribution scope checks use QMSum [Zhong et al., 2021], MSC persona [Xu et al., 2022], HotpotQA [Yang et al., 2018], and MuSiQue [Trivedi et al., 2022].

3 ConvMemory Architecture

3.1 Configuration

Table 1 summarizes the distributed ConvMemory checkpoint. The total parameter count is approximately 3.6M.

Table 1: ConvMemory checkpoint configuration (the distributed LoCoMo MPNet checkpoint).

Field	Value
Embedding backbone	sentence-transformers/all-mpnet-base-v2
Embedding dimension	768
Window size	5
Stride	1
Kernel size	3
Hidden dimension	256
Token MLP dim	32
Channel MLP dim	512
Candidate top- n	500
Raw score fusion weight	0.025
Total parameters	$\sim 3,648,118$

3.2 Components

ConvMemory composes four components into a single CE-lite scoring function for each candidate in the dense retriever’s top-500 pool.

1. **Dense candidate scoring.** The base sentence encoder scores query-candidate cosine similarity over the memory store; the top-500 candidates by dense score form the pool.
2. **Sliding-window conv/mixer encoder.** A small windowed encoder (window size 5, stride 1, kernel size 3, hidden dim 256, token-MLP 32, channel-MLP 512) processes the candidate stream within the top-500 pool. *The window operates over the candidate ranking order (sorted by dense score), not over chronological memory order.* Early versions of this work conjectured that, because candidate ranking and memory time are correlated in conversational data, the windowed encoder would indirectly capture temporal structure. Section 5 shows that this interpretation does not survive a proper attribution study.
3. **Lexical interaction features.** Query-candidate lexical features (token overlap and related) provide a cheap, discriminative signal that complements the dense channel.
4. **Optional block-level router scalar.** A learned scalar gates between sub-features. Its contribution is approximately zero in the retrained ablation (Section 5).

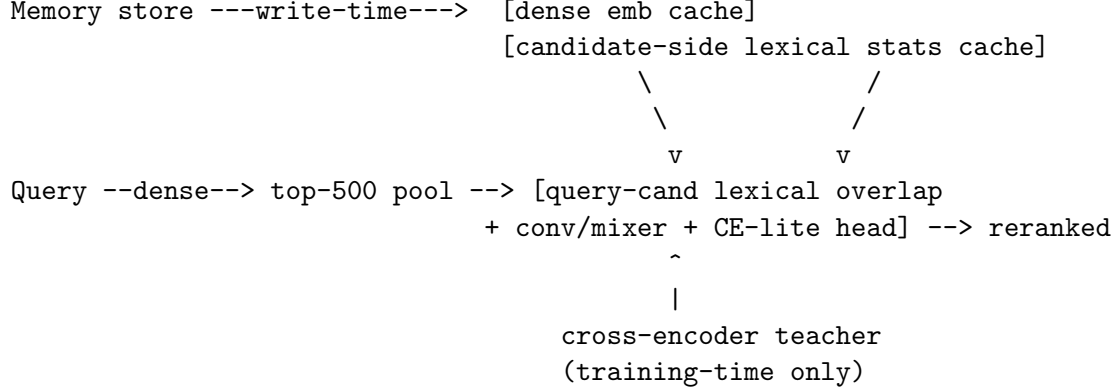
3.3 Training objective

ConvMemory is trained with cross-encoder teacher supervision on LoCoMo top-500 candidate pools. The teacher is `cross-encoder/ms-marco-MiniLM-L-6-v2`. We deliberately do not match the teacher to the strongest available cross-encoder (`mxbai-rerank-large-v1`); doing so would conflate distillation gains with teacher choice and weaken the cost-frontier comparison. The objective combines a pairwise ranking loss over candidate pairs with a first-rank objective that pushes the gold memory to rank 1.

3.4 Cost structure

The structural source of ConvMemory’s cost advantage is feature precomputability on the candidate side. Dense embeddings and candidate-side lexical statistics (e.g., candidate tokenization, IDF-weighted term sets) are computed at memory-write time and cached. The truly query-dependent lexical interaction features (e.g., token overlap, query-conditioned weighting) are computed at inference time, but cheaply: they are sparse set operations over the cached

candidate-side tokenization, not transformer forward passes. The remaining inference work is a small per-query fusion — the conv/mixer encoder over the 500-candidate window and the CE-lite head. The cross-encoder baselines we compare against, in contrast, must run a full transformer forward over each query-candidate pair at query time, which cannot be amortized across queries.



We do not claim this architecture is novel; the contribution is the cost-quality operating point this particular combination delivers on conversational memory retrieval, and the careful attribution study of why it works.

4 Main Results

4.1 LongMemEval cost advantage

Table 2 reports Recall@10, MRR, and per-query latency on LongMemEval. Two settings are reported: Clean500 (no distractor stress) and Stress1000 with seed 23 (1000 distractor memories per question).

Table 2: LongMemEval cost-frontier measurements. Clean500 is single-run; Stress1000 is single-seed (seed 23). Indicative cost-frontier evidence, not benchmark-grade.

Setting	Method	Recall@10	MRR	ms/query
Clean500, BGE-large CE	Raw MPNet	0.9049	0.7829	0.01
Clean500, BGE-large CE	BGE-large CE top500	0.8807	0.8574	555.69
Clean500, BGE-large CE	ConvMemory top500	0.9593	0.8973	44.00
Clean500, mxbai CE	mxbai CE top500	0.9835	0.9317	1129.14
Clean500, mxbai CE	ConvMemory top500	0.9593	0.8973	40.80
Stress1000 s23, BGE CE	BGE-large CE top500	0.6913	0.6651	5231.77
Stress1000 s23, BGE CE	ConvMemory cand-local	0.7386	0.6125	110.71
Stress1000 s23, mxbai CE	mxbai CE top500	0.8195	0.7044	11211.63
Stress1000 s23, mxbai CE	ConvMemory cand-local	0.7386	0.6125	95.57

On Clean500, ConvMemory has higher Recall@10 than the BGE-large cross-encoder (0.9593 vs 0.8807) at $555.69/44.00 \approx 12.6\times$ lower latency, and stays within 0.025 Recall@10 of mxbai-rerank-large-v1 (0.9593 vs 0.9835) at $1129.14/40.80 \approx 27.7\times$ lower latency. Under Stress1000 distractors, ConvMemory beats the BGE-large cross-encoder on Recall@10 (0.7386 vs 0.6913) at $5231.77/110.71 \approx 47.3\times$ lower latency. Against mxbai-rerank-large-v1 under Stress1000, ConvMemory’s Recall@10 drops 0.081 below mxbai (0.7386 vs 0.8195), but it runs $11211.63/95.57 \approx 117.3\times$ cheaper; we report this trade-off explicitly rather than presenting only the favorable Clean500 comparison.

Latency measurement protocol (summary). All ms/query figures are reranking-only wall-clock time over the top-500 candidate pool, measured on a single NVIDIA GeForce RTX 4080 SUPER (32GB). Cross-encoder batch sizes are tuned per model to a throughput-favourable setting (32–128 query-candidate pairs per forward, larger for smaller models); ConvMemory’s per-query fusion runs at query batch size 1 with GPU-vectorized scoring of the 500-candidate window. Dense embedding computation, candidate-side feature precomputation, and dense retrieval are excluded from all rows — they are shared across methods and would only shift all numbers by the same constant. The Raw MPNet 0.01 ms/query figure reflects only the cost of sorting precomputed dense scores; it is included as a lower bound, not as a comparable method. Full protocol, including hardware specification, software stack, per-model batching configuration, and what is and is not included in each measurement, is in Appendix A.

What this is not evidence for. These LongMemEval numbers are single-run (Clean500) or single-seed (Stress1000 seed 23). They are intended as a cost-frontier indicator on memory-style retrieval, not as a benchmark-grade comparison. On MRR specifically, ConvMemory loses to both cross-encoders on Stress1000.

4.2 LoCoMo strong cross-encoder baselines

Table 3 reports a 5-seed comparison against strong cross-encoder rerankers on LoCoMo, with seeds 7, 11, 23, 31, 47 and the dense retriever’s top-500 as input pool.

Table 3: LoCoMo strong reranker baselines (5 seeds: 7, 11, 23, 31, 47; raw dense top-500 pool). All entries are 5-seed mean $\pm$ std. The ConvMemory v0.40 row aggregates per-seed metrics from the archived v0.40 evaluation; earlier public README summaries conservatively omitted the Hit@10 and MRR variance, but the per-seed CSV retains the full distribution and is the [source for this row](#).

Reranker	Recall@10	Hit@10	MRR
ConvMemory (v0.40, 5 seeds)	0.7798 $\pm$ 0.0074	0.8350 $\pm$ 0.0083	0.5824 $\pm$ 0.0189
BGE-reranker-base	0.6967 $\pm$ 0.0126	0.7469 $\pm$ 0.0144	0.5469 $\pm$ 0.0140
Jina-reranker-v2-base-multilingual	0.7411 $\pm$ 0.0103	0.7924 $\pm$ 0.0083	0.5754 $\pm$ 0.0074
BGE-reranker-large	0.7621 $\pm$ 0.0155	0.8124 $\pm$ 0.0135	0.6120 $\pm$ 0.0144
mxbai-rerank-large-v1	0.8080 $\pm$ 0.0153	0.8486 $\pm$ 0.0108	0.6687 $\pm$ 0.0093

Honest reading. ConvMemory is competitive with the BGE-reranker family on Recall@10 (above BGE-reranker-base and BGE-reranker-large in mean Recall@10 while being one to two orders of magnitude smaller in parameter count depending on the baseline) but *loses* to mxbai-rerank-large-v1 on both Recall@10 and MRR. We do not aggregate ConvMemory and CCGE-LA gains together to argue against mxbai: even with the CCGE-LA editor on top, the absolute MRR remains well below mxbai-rerank-large-v1.

4.3 Strong-backbone retraining

Table 4 reports a 3-seed retraining study with stronger embedding backbones (BGE-large and E5-large) in place of MPNet.

Reading. Gains generalize across embedding backbones. **Caveat:** 3-seed study, and the BGE-large / E5-large ConvMemory checkpoints are reported here but not currently distributed via the Hugging Face Hub.

Table 4: Strong-backbone retraining on LoCoMo (3 seeds). The +0.09–0.10 Recall@10 lift survives when the backbone is swapped.

Backbone	Raw R@10	ConvMemory R@10	Gain	ConvMemory MRR
BGE-large	0.6680 $\pm$ 0.0237	0.7726 $\pm$ 0.0100	+0.1046 $\pm$ 0.0137	0.5639 $\pm$ 0.0066
E5-large	0.7010 $\pm$ 0.0216	0.7902 $\pm$ 0.0171	+0.0892 $\pm$ 0.0052	0.5941 $\pm$ 0.0103

5 Rigorous Attribution: A Negative Result

This section is the headline scientific contribution of the report. We begin by stating the original conjecture, then present the evidence that refutes it.

Original conjecture. As stated in Section 3, the windowed conv/mixer encoder operates over candidate *ranking* order, not chronological order. Early versions of this work conjectured that, because dense-rank neighborhoods on conversational memory data are correlated with temporal locality (recent and topically-similar memories tend to cluster together in the dense ranking), the windowed encoder would *indirectly* learn temporal structure useful for memory retrieval, and that this implicit temporal capture was the load-bearing reason ConvMemory worked on T_SUP and T_HOP query slices. The evidence below shows this interpretation does not survive a careful study: the window helps, but not on the slices a temporal mechanism would help on.

5.1 Retrained ablation (3 seeds)

Table 5 reports a 3-seed retrained ablation on MPNet. Each variant is retrained from scratch with the corresponding component removed.

Table 5: Retrained ablation, 3 seeds, MPNet backbone. Lexical features dominate; the temporal window is smaller; the router is approximately zero.

Variant	Recall@10	MRR	Δ R@10 vs full
full_control	0.7474 $\pm$ 0.0229	0.5343 $\pm$ 0.0160	0.0000
no_router	0.7491 $\pm$ 0.0213	0.5391 $\pm$ 0.0137	+0.0017 $\pm$ 0.0020
no_temporal_w1	0.7121 $\pm$ 0.0232	0.5305 $\pm$ 0.0148	−0.0353 $\pm$ 0.0052
no_lexical	0.6584 $\pm$ 0.0185	0.4367 $\pm$ 0.0129	−0.0890 $\pm$ 0.0061
no_lexical_no_router	0.6574 $\pm$ 0.0163	0.4342 $\pm$ 0.0127	−0.0899 $\pm$ 0.0087

The lexical features contribute the largest delta (−0.089 Recall@10 when removed). The temporal window contributes a smaller but non-trivial delta (−0.035). The router scalar contributes approximately zero (+0.0017 $\pm$ 0.0020). This alone is insufficient to claim what the temporal contribution *is*, only that it is not negligible on aggregate.

5.2 Five-seed retrained attribution with paired bootstrap

We then ran the v0.51 5-seed retrained attribution and computed paired bootstrap confidence intervals for the full_control – no_temporal_w1 delta on Recall@10, sliced by query type.

Mean Recall@10 (5 seeds): full_control_retrained 0.7432 $\pm$ 0.0207, no_temporal_w1_retrained 0.7054 $\pm$ 0.0221, tuned_heuristic (dense+BM25+temporal+decay) 0.7234 $\pm$ 0.0227, raw_dense 0.5345 $\pm$ 0.0210.

Additionally, full_control – tuned_heuristic on ALL Recall@10 is +0.0199 [+0.0105, +0.0283], and the corresponding MRR delta is +0.0566.

Table 6: Paired bootstrap deltas, full_control – no_temporal_w1, Recall@10. The aggregate delta is significant on all slices except T_HOP_auto; the rank ordering across slices is the reverse of what a temporal-specific mechanism would predict (see Section 5 Interpretation).

Slice	Δ	95% CI	Reading
ALL	+0.0376	[+0.0306, +0.0451]	significant
T_SUP_auto	+0.0407	[+0.0219, +0.0603]	significant
T_REQUIRED_auto	+0.0252	[+0.0139, +0.0363]	significant
T_HOP_auto	+0.0096	[−0.0037, +0.0230]	NOT significant
OTHER	+0.0868	[+0.0672, +0.1045]	significant (largest)
HARD_NON_TEMPORAL_auto	+0.0838	[+0.0650, +0.1040]	significant

Interpretation. The aggregate gain attributed to the window is statistically significant, and on individual slices the window does help on T_SUP_auto (+0.0407) and T_REQUIRED_auto (+0.0252); these are significant and are by themselves compatible with some temporal-relevant signal. The slice pattern as a whole, however, does not match what a temporal-specific mechanism would predict. A genuinely temporal mechanism should produce its largest effects on the most temporally-demanding slice (multi-hop temporal reasoning, T_HOP_auto) and smaller or negligible effects on hard non-temporal controls. The observed pattern is the reverse rank ordering: the effect is *largest* on HARD_NON_TEMPORAL_auto (+0.0838) and OTHER (+0.0868), and is *not* statistically significant on T_HOP_auto (+0.0096, CI crosses zero).

The fact that the window helps *more* on hard non-temporal queries than on temporal-hop queries means the mechanism is not uniquely or specifically temporal. The most parsimonious description is that the windowed encoder operates as generic neighborhood / capacity smoothing in the fused dense+lexical feature space, with T_SUP remaining an open slice where some temporal-relevant signal may still be present but cannot be cleanly attributed to the window from this experiment alone. The overall mechanism best fits cheap cross-encoder distillation in the fused dense+lexical feature space, with the windowed encoder providing modest extra capacity that helps broadly on hard queries.

Why publish this. We publish this negative result on a previously claimed mechanism because the discipline matters. The engineering value of ConvMemory (cost-effective reranking at a useful operating point) is independent of the mechanism story and survives the negative attribution.

What this is not evidence for. This negative result is about *ConvMemory’s* temporal window on *LoCoMo*. It does not rule out the possibility that other temporal mechanisms in other architectures on other benchmarks are load-bearing. It also does not rule out the possibility that a different temporal mechanism, properly designed, would help on T_HOP.

6 CCGE-LA: Research-Preview Conflict Editor

CCGE-LA (“Low-Amplitude Counterfactual Conflict Graph Editor”) is a tiny editor over ConvMemory’s candidate set, released as a research preview.

Motivation. As shown later in Section 7, small from-scratch stream rerankers on real LoCoMo fail without a teacher signal. CCGE-LA takes the opposite approach: rather than replacing ConvMemory with a new from-scratch model, it reads ConvMemory’s candidate set and applies a small residual correction when the candidate set looks conflict-prone (e.g., when a stale and a current memory both score highly).

6.1 Architecture

CCGE-LA computes 18 candidate-set features for each candidate in the top- k pool:

```
base_score_z,      dense_score_z,      position_z,      query_overlap_z,  
base_rank_norm,   dense_rank_norm,   sim_to_base_top,   sim_to_dense_top,  
semantic_density_top16, token_overlap_to_top, newer_than_base_top,  
older_than_base_top, abs_pos_gap_top_z, base_margin_1_2, base_entropy_top16,  
conflict_density_top16, time_span_top16, top_overlap.
```

These features are passed through a small transformer encoder, then a residual head produces a per-candidate score correction. A sigmoid gate with negative-bias initialization controls editor amplitude, so an untrained editor is approximately a no-op (gate close to 0). This is a deliberate safety property: the editor cannot make things much worse before it has learned anything.

6.2 Training

Training uses retrieval cross-entropy only. We explicitly enforce three anti-shortcut rules:

- No current/stale labels in features. The editor does not get to peek at which candidate is canonical.
- No gold-defining feature. No feature uniquely identifies the gold answer.
- No distillation as the defining mechanism. The editor is not trained against a stronger teacher; it must learn from retrieval supervision alone.

6.3 Results

Table 7: CCGE-LA per-slice best from internal 5-seed exploration on real LoCoMo. Each row reports the strongest internal editor variant on that specific slice; the variants differ across rows (FULL is best under V150, T_SUP_auto under V145, the rescue slices under V149). This is an internal upper-bound frontier across our exploration, not the behavior of a single deployable model. The released alpha checkpoint, which is a single variant and a single seed, is reported separately in Table 8 and underperforms this per-slice frontier by design.

Setting	ConvMemory MRR	Best editor MRR	Δ
FULL	0.5708	0.5878	+0.0170
T_SUP_auto	0.5518	0.5803	+0.0285
CONV_TOP1_WRONG_GOLD_IN_POOL	0.2586	0.3285	+0.0699
RESCUABLE_STALE_TOP1	0.2636	0.3655	+0.1019

The lifts in Table 7 are not the behavior of a single CCGE-LA model. We ran multiple editor variants during internal exploration; different slices were strongest under different variants. Table 7 reports the per-slice best across this exploration. Read this way, the per-slice frontier shows: on FULL MRR the best variant (V150) lifts +0.0170 over ConvMemory base; on the supersession slice (T_SUP_auto) the best variant (V145) lifts +0.0285; on diagnostic conflict slices the best variants (V149) lift +0.0699 and +0.1019. The released alpha checkpoint (Table 8) is a single variant and seed and lifts a smaller, uniform amount across slices. We present the per-slice frontier as evidence that conflict-aware editing carries useful signal under our slice diagnostics; we do not claim a single deployable model matches the frontier.

Table 8: CCGE-LA published alpha checkpoint (single seed, seed 23). The distributed checkpoint underperforms the best 5-seed editor variant; the 5-seed numbers in Table 7 should be read as aggregate gains, not as the released checkpoint’s behavior.

Subset	MRR	R@10	gate
FULL	0.5638	0.7725	0.0995
T_SUP_auto	0.5508	0.7138	0.0995
CONV_TOP1_WRONG_GOLD_IN_POOL	0.2994	0.6822	0.0995
RESCUABLE_STALE_TOP1	0.3093	0.6877	0.0995

Honest framing. Modest aggregate lift; larger lift on supersession and rescue slices. The published alpha checkpoint is single-seed. **This is explicitly a research preview / alpha.** The limits are: single task family (LoCoMo conversational memory), single backbone (MPNet), no broad cross-task validation. CCGE-LA is listed in this report as a research-preview contribution (see Section 1); we do not claim it as a general conflict-resolution mechanism or as equal in maturity to the engineering and attribution contributions.

7 Real-Data Stream-Reranker Negative Result (v0.59)

A natural counter-question to ConvMemory is: do you really need a cross-encoder teacher, or can a small stream-architecture reranker learn this directly from retrieval supervision? We tested both a recurrent running-state reranker and a flat transformer over the memory stream, both trained from scratch on real LoCoMo with retrieval-only supervision. To our knowledge, this specific negative result has not been published before; it is reported here for the first time.

Table 9: v0.59 real-LoCoMo stream-reranker study, 5-seed full run. Main finding: both from-scratch stream rerankers fail below raw dense on the supersession slice; only the cross-encoder-distilled ConvMemory checkpoint beats raw dense. Cost ratio = inference time at large N / inference time at small N ; the recurrent state is structurally roughly flat in N by construction. Full latency protocol in Appendix A.

Arm	T_SUP R@10	T_SUP MRR	FULL R@10	FULL MRR	params	cost ratio
raw_dense	0.5005 $\pm$ 0.0496	0.2939 $\pm$ 0.0267	0.5345 $\pm$ 0.0210	0.3254 $\pm$ 0.0105	0	484.50 $\times$
recency_rule	0.5275 $\pm$ 0.0398	0.4109 $\pm$ 0.0477	0.5318 $\pm$ 0.0083	0.4108 $\pm$ 0.0138	0	49.78 $\times$
strong_reranker_ref	0.7061 $\pm$ 0.0306	0.5396 $\pm$ 0.0293	0.7498 $\pm$ 0.0080	0.5610 $\pm$ 0.0116	3,648,118	7.99 $\times$
transformer_over_stream	0.3127 $\pm$ 0.0548	0.1681 $\pm$ 0.0191	0.3638 $\pm$ 0.0534	0.1927 $\pm$ 0.0245	165,089	10.21 $\times$
running_state_gru	0.2826 $\pm$ 0.0483	0.1379 $\pm$ 0.0236	0.3474 $\pm$ 0.0339	0.1787 $\pm$ 0.0198	161,562	1.60 $\times$
running_state_gru_large	0.2953 $\pm$ 0.0493	0.1503 $\pm$ 0.0300	0.3414 $\pm$ 0.0185	0.1831 $\pm$ 0.0092	928,929	1.07 $\times$

The **strong_reranker_ref** arm is the ConvMemory checkpoint. “Cost ratio” is the inference cost at large N relative to small N (lower is better; recurrent state is structurally roughly flat in N).

Reading. Both from-scratch stream rerankers fail below raw dense on the supersession slice (T_SUP R@10 0.31 and 0.28 vs raw dense 0.50). Increasing capacity from 161K to 929K parameters does not rescue them. The recurrent state’s cost asymmetry (roughly flat in N) is structurally real, but combined with the effectiveness failure it supports the conclusion that the teacher signal source is necessary at this data and parameter scale. ConvMemory’s value is not an arbitrary stream-architecture effect.

What this is not evidence for. It does not rule out larger-data or larger-parameter regimes where a from-scratch stream reranker might succeed. It rules out the small-budget, retrieval-

only-supervised, real-LoCoMo regime studied here.

8 Synthetic Benchmark Trajectory: A Methodology Lesson

Across multiple internal synthetic benchmarks (versions roughly v0.52 through v1.42), benchmark design was repeatedly degenerate. In each iteration, the gold answer turned out to be recoverable by a trivial baseline: recency, position-equals-time, or oracle-equivalent supervised detectors. The cleanest current benchmark is V142, which was externally verifier-audited per `docs/RESEARCH_TRAJECTORY.md` in the public repository.

Even on V142, the benchmark decomposes into two subsets that must be reported separately.

Table 10: V142 synthetic, current subset (no date in the query). On this subset, bitemporal is mathematically identical to `event_time_no_asof`, because the as-of filter is skipped when the query has no date. The real mechanism on current queries is “learned support extraction + event-time ordering.”

Arm	R@1	MRR	stale@1
support_only	0.4660	0.6909	0.3990
event_time_no_asof	0.7360	0.8539	0.0700
write_order	0.7090	0.8123	0.1290
bitemporal	0.7360	0.8539	0.0700

Table 11: V142 synthetic, as-of subset (date in the query; the query embeds the target date). The as-of gain depends on a hand-designed bitemporal ranking rule that uses the query-embedded date. This is not a learned mechanism on this subset.

Arm	R@1	MRR	stale@1
support_only	0.6675	0.8025	0.3325
event_time_no_asof	0.1450	0.5625	0.8550
write_order	0.3025	0.5771	0.6975
bitemporal	0.9475	0.9738	0.0525

The honest claim is two separate results, not one aggregate “bitemporal +0.27” gain. The bitemporal arm consists of a learned operation/support extractor plus a hand-designed bitemporal ranking rule; we deliberately do not call it “a learned bitemporal mechanism.”

Methodology lesson. Cleanly attributing a mechanism on synthetic memory tasks is structurally harder than it looks. Gold definitions on memory tasks frequently collapse into trivial-baseline-output by construction, because the same generative process that creates the questions also creates an oracle the trivial baseline can exploit. The community should expect this failure mode and design accordingly: cross-check every synthetic gain against a recency, position, and oracle-detector baseline before claiming a mechanism.

9 External OOD Scope Check

Table 12 reports single-run results on four external out-of-distribution tasks.

ConvMemory wins on QMSum (0.5882 vs the next-best 0.5423 dense+lexical). On MSC persona it strongly improves over raw dense (0.9632 vs 0.8375) but is dominated by both dense+lexical (0.9765) and BM25 (0.9920). On HotpotQA it improves over raw dense but

Table 12: External OOD scope check (single run each, indicative). ConvMemory is a conversational-memory reranker; these tasks span very different distributions.

Dataset	Q	ConvMemory R@10	Raw dense	Dense + lexical	BM25
QMSum	272	0.5882	0.4724	0.5423	0.5294
MSC persona	6155	0.9632	0.8375	0.9765	0.9920
HotpotQA	1000	0.7983	0.7682	0.8621	0.8280
MuSiQue	1000	0.7635	0.8640	0.8175	0.7245

is below dense+lexical (0.7983 vs 0.8621). On MuSiQue it regresses below raw dense (0.7635 vs 0.8640).

Reading. ConvMemory is a conversational-memory reranker. We do not claim it as a general multi-hop document reranker; the MuSiQue regression is consistent with that scope. We do claim that on conversation-like distributions (QMSum, MSC persona vs raw dense) it carries useful signal.

What this is not evidence for. Single-run indicative scope checks; not benchmark-grade.

10 Limitations

- Only the LoCoMo MPNet base checkpoint is distributed via Hugging Face. The BGE-large and E5-large gains in Table 4 are reported but the corresponding checkpoints are not currently released.
- The CCGE-LA distributed checkpoint is single-seed (seed 23). The aggregate gains in Table 7 rest on the 5-seed internal study; the distributed checkpoint should be treated as an alpha.
- No end-to-end agent or QA task evaluation. All results are retrieval-stage. We do not measure downstream task performance.
- ConvMemory does not beat mxbai-rerank-large-v1 in absolute LoCoMo MRR; CCGE-LA on top does not close that gap.
- The V142 synthetic as-of subset gain depends on a query-embedded target date. It should not be read as proof of an unconditional bitemporal mechanism.
- LongMemEval numbers are single-run (Clean500) or single-seed (Stress1000 seed 23).
- External OOD results in Table 12 are single-run indicative scope checks, not benchmark-grade comparisons.
- The “honest negative result” framing is single-author; it has not yet been independently audited.

11 Discussion and Future Work

Three directions follow naturally from the limitations.

Multi-backbone checkpoint distribution. The strong-backbone retraining gains (BGE-large, E5-large) are reported but those checkpoints are not currently distributed. Releasing them would let practitioners pick the cost-quality operating point that matches their existing embedding stack.

End-to-end agent benchmark. The cost-frontier evidence in Section 4.1 is suggestive but retrieval-stage. Integrating ConvMemory + CCGE-LA into a full agent pipeline (retrieval → LLM answerer) and measuring downstream QA quality would test whether the retrieval-stage gains translate to user-visible improvements.

Broader CCGE-LA training. CCGE-LA is currently single-task (LoCoMo), single-backbone (MPNet), and single-seed in its distributed form. Multi-task, multi-seed training, with attention to whether the editor’s gains transfer across distributions, is required before it can be promoted from research preview.

12 Reproducibility

The public repository is at <https://github.com/pth2002/ConvMemory>. The two Hugging Face model repositories are:

- <https://huggingface.co/Purdy0228/ConvMemory-LoCoMo-MPNet> – the distributed ConvMemory checkpoint.
- <https://huggingface.co/Purdy0228/ConvMemory-CCGE-LA> – the alpha CCGE-LA editor checkpoint (single seed, seed 23).

Exact reproduction commands are in `docs/EVALUATION_PROTOCOL.md`. Per-question CSVs and embedding caches are intentionally not redistributed; reproduction commands regenerate them from the public benchmark sources.

A Full Latency Protocol

This appendix documents the latency measurement protocol underlying Table 2 in Section 4.1 and the cost-ratio column of Table 9 in Section 7. The intent is not to publish benchmark-grade latency numbers; the LongMemEval rows are single-run (Clean500) or single-seed (Stress1000 seed 23). The intent is to make the cost-frontier comparison reproducible and to document what each ms/query figure does and does not include.

Hardware. All latency measurements were run on a single NVIDIA GeForce RTX 4080 SUPER (32GB) GPU. The host node has Intel Xeon Platinum 8352V CPUs (2 sockets, 32 cores per socket, 128 hardware threads total). No multi-GPU or distributed inference was used. No CPU-only configuration was measured.

Software stack. Python 3.12.3, PyTorch 2.5.1 with CUDA 12.1 (`torch==2.5.1+cu121`), HuggingFace `transformers` 4.46.3, and `sentence-transformers` 3.3.1 for MPNet embeddings. The dense top-500 retrieval step uses NumPy cosine similarity / matrix dot product followed by `argsort`; no approximate-nearest-neighbour index (FAISS, HNSW, etc.) was used. This means our dense-retrieval cost is conservative on the upper end — a production deployment with an ANN index would be cheaper, not more expensive — and the cross-encoder vs. ConvMemory comparison is not sensitive to this choice because dense retrieval is shared and excluded from all reranking ms/query figures.

Batching. Cross-encoder baselines on the LoCoMo 5-seed run (Section 4.2) used the following per-model batch sizes (query-candidate pairs per forward), tuned to fit the 32GB GPU memory budget without OOM while staying close to throughput-optimal for each model size:

- **cross-encoder/ms-marco-MiniLM-L-6-v2:** batch size 128 (used as the training teacher signal source; not a comparison baseline in LongMemEval).
- **BAAI/bge-reranker-base:** batch size 64.
- **BAAI/bge-reranker-large:** batch size 32.
- **mxbai-rerank-large-v1:** batch size 32.
- **jinaai/jina-reranker-v2-base-multilingual:** batch size 32.

LongMemEval runs use a default cross-encoder batch size of 64 on Clean500. The Stress1000 seed 23 runs use larger model-specific batches as recorded in the archived logs (BGE-large at batch 128, mxbai at batch 96 or 64 depending on memory headroom). ConvMemory itself runs at query batch size 1; within each query, the conv/mixer encoder and the CE-lite head score the full 500-candidate window in a single GPU-vectorized forward pass, which is its natural operating regime.

What is included in each ms/query figure.

- **Raw MPNet (0.01 ms/query).** Only the cost of sorting precomputed dense scores to produce a top-10 list. This is included as a lower bound on what is structurally achievable when reranking is skipped entirely. It is *not* a method we compare against; the dense embedding computation and top-500 retrieval are not counted.
- **Cross-encoder baselines (BGE-large CE, mxbai CE, etc.).** Per-query wall-clock time to score 500 query-candidate pairs through the cross-encoder forward pass and sort them, given the top-500 candidate pool. Excludes the dense retrieval step that produced the top-500.
- **ConvMemory (top500, cand-local).** Per-query wall-clock time of the conv/mixer encoder forward over the 500-candidate window plus the CE-lite head, given the top-500 candidate pool and given that candidate-side lexical statistics have been precomputed and cached. Excludes the dense retrieval step. The **top500** and **cand-local** variants differ in how the candidate-side cache is structured; both exclude the same shared upstream work.

What is excluded from all rows. Dense embedding computation, dense retrieval (top-500 construction), CPU-to-GPU tensor transfer at startup, model loading, and warm-up. All measurements are reported after a warm-up pass to avoid first-call CUDA initialization overhead. Each ms/query figure is the mean over the full LongMemEval evaluation set in the corresponding row: 500 queries on Clean500 and 1000 queries on Stress1000 seed 23.

Why these exclusions are fair, and one caveat. Dense embedding computation and dense retrieval (top-500 construction) are genuinely shared upstream cost across all methods — they are part of the pipeline upstream of any reranker. Including them would add the same constant to every row and would not change the cost-ratio claims. Excluding them isolates the reranking-stage cost, which is what this report is making claims about.

Candidate-side lexical statistics are different: they are *not* shared. They are a ConvMemory-specific cost, paid once per memory at memory-write time and amortized across all future queries that read the memory store. Our latency protocol excludes this write-time cost from per-query

measurements because it does not scale with query volume in a low-write/high-read regime, which is the typical agent-memory deployment. Practitioners with high write volume (e.g., agents that rapidly ingest new memories during a session) should add the lexical-cache write cost to their own deployment budget when comparing against cross-encoder baselines. We do not claim end-to-end pipeline latency; we claim reranking-stage cost-frontier evidence under the stated regime.

Cost ratio definition (used in Table 9). The cost ratio column in the v0.59 stream-reranker table is inference time at large N (full memory store) divided by inference time at small N (early-conversation memory store). A ratio close to 1 means the method’s cost is roughly flat in memory-store size; large ratios mean the method scales poorly with N . The recurrent state arms have ratios close to 1 by construction (their per-query cost depends only on the running state, not on the full memory store); the cross-encoder-style arms have ratios that grow with N because they score every candidate against the query.

Known caveats.

- All numbers are single-run (Clean500) or single-seed (Stress1000 seed 23). Latency has run-to-run noise on shared hardware; we do not report variance bars.
- Per-query timing on a single RTX 4080 SUPER may not reflect the cost frontier on data-centre GPUs (e.g., A100, H100), on smaller GPUs (e.g., T4, L4), on CPU-only inference, or on inference servers with kernel fusion (e.g., TensorRT, vLLM). Practitioners should benchmark on their target hardware.
- The cross-encoder baselines are evaluated as off-the-shelf checkpoints with default tokenization and sequence length (typically 512 tokens). Aggressive truncation or distillation of these baselines would change the comparison.
- Cross-encoder batch sizes were chosen to fit the 32GB GPU memory budget without OOM. On larger-memory GPUs, larger batches would reduce cross-encoder ms/query somewhat; this would narrow but not close the ConvMemory cost gap, since ConvMemory’s per-query work is already an order of magnitude smaller in FLOPs.

References

- Jina AI. jina-reranker-v2-base-multilingual. Hugging Face Model Hub, 2024. URL <https://huggingface.co/jinaai/jina-reranker-v2-base-multilingual>. Accessed: 2026-05-21.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL <https://arxiv.org/abs/2402.17753>.
- Mem0 Team. Mem0: Long-term memory for AI agents. GitHub repository, 2024. URL <https://github.com/mem0ai/mem0>. Accessed: 2026-05-21.
- Mixedbread AI. mxbai-rerank-large-v1. Hugging Face Model Hub, 2024. URL <https://huggingface.co/mixedbread-ai/mxbai-rerank-large-v1>. Accessed: 2026-05-21.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL <https://arxiv.org/abs/2310.08560>.

- Nils Reimers and Iryna Gurevych. cross-encoder/ms-marco-MiniLM-L-6-v2. Hugging Face Model Hub, 2021. URL <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L-6-v2>. Accessed: 2026-05-21.
- Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. MPNet: Masked and permuted pre-training for language understanding. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL <https://arxiv.org/abs/2004.09297>.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 2022.
- Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022. URL <https://arxiv.org/abs/2212.03533>.
- Di Wu, Hongwei Wang, Wenhao Yu, Yunsheng Zhang, Kai-Wei Chen, and Dong Yu. Long-MemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024. URL <https://arxiv.org/abs/2410.10813>.
- Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*, 2023. URL <https://arxiv.org/abs/2309.07597>.
- Jing Xu, Arthur Szlam, and Jason Weston. Beyond goldfish memory: Long-term open-domain conversation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.
- Ming Zhong, Da Yin, Tao Yu, Ahmad Zaidi, Mutethia Mutuma, Rahul Jha, Ahmed Hassan Awadallah, Asli Celikyilmaz, Yang Liu, Xipeng Qiu, and Dragomir Radev. QMSum: A new benchmark for query-based multi-domain meeting summarization. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2021.